Soft Computing

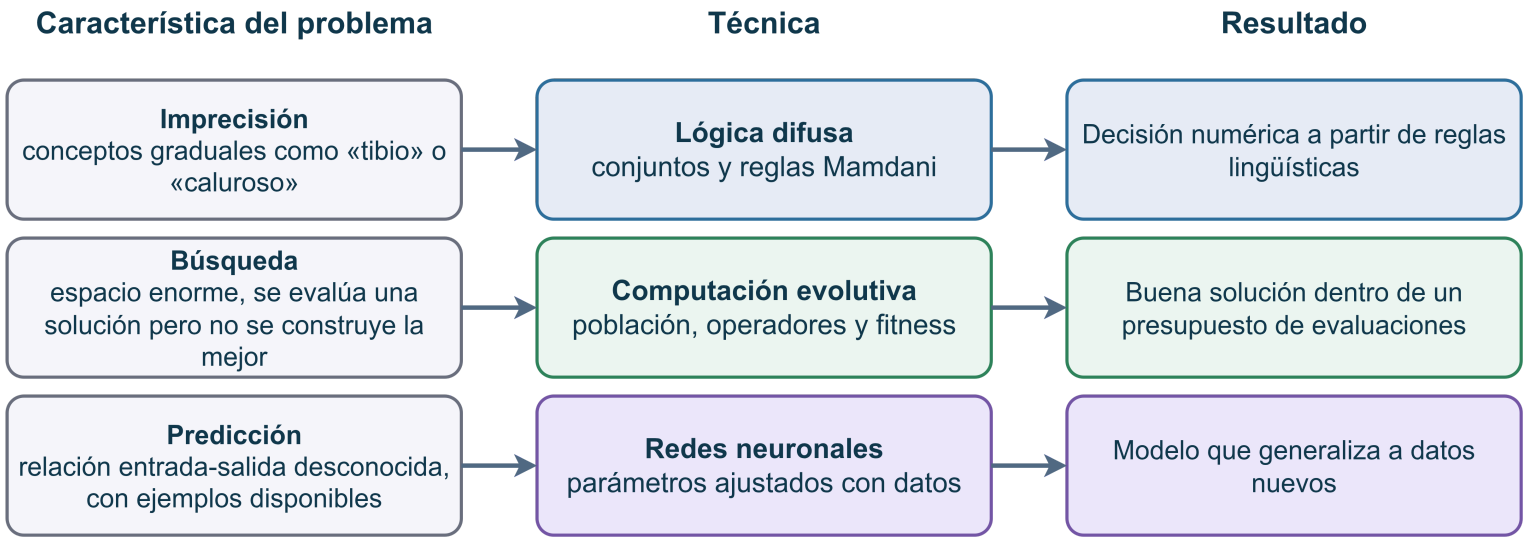
Repaso de complejidad, sistemas difusos, computación evolutiva y redes neuronales

Complejidad y búsqueda aproximada

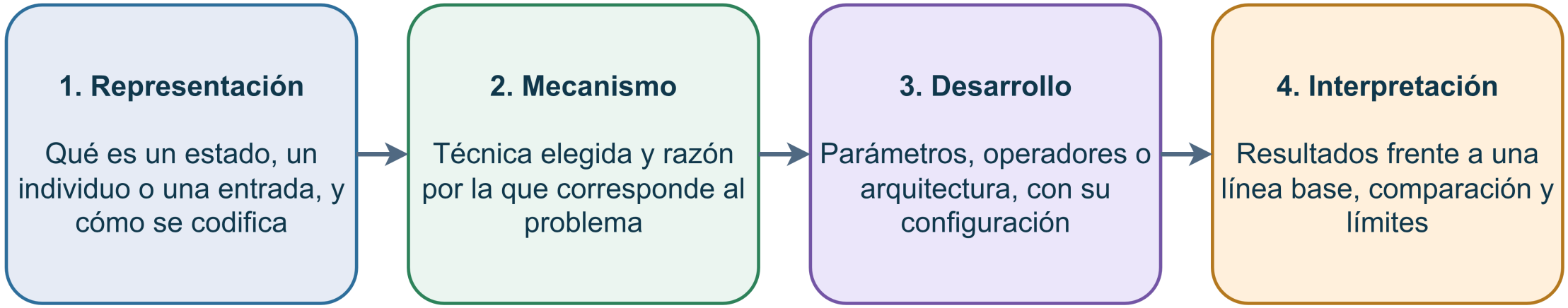
Cuándo una solución exacta es inviable

Soft Computing: soluciones aproximadas

> **Soft Computing** reúne técnicas que entregan soluciones aproximadas, pero útiles, cuando la solución exacta es muy costosa o no se puede calcular.



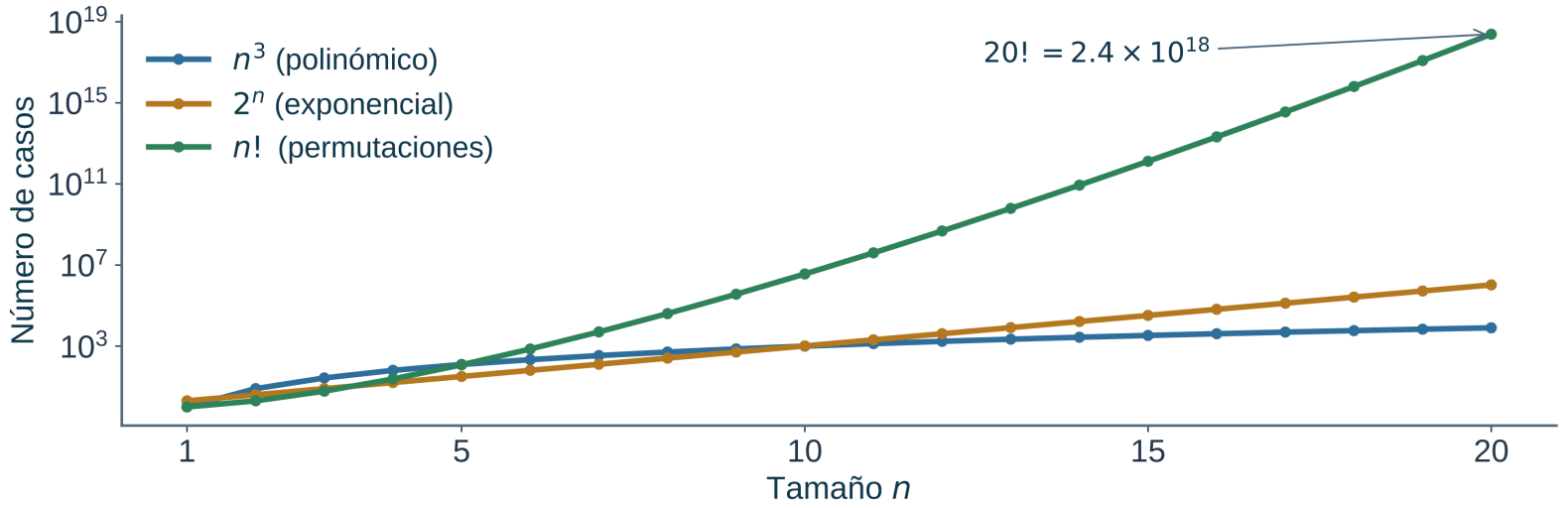
Cómo se describe una solución



Cada técnica se explica con las mismas cuatro partes. La interpretación compara el resultado con una referencia y declara sus límites.

Espacio de búsqueda y su crecimiento

Un **estado** es una configuración posible del problema. El **espacio de búsqueda** S reúne todos los estados y su tamaño es $|S| = k^n$, donde k es la cantidad de valores por posición y n la cantidad de posiciones. En tres en línea, con $k = 3$ y $n = 9$, se tiene $|S| = 3^9 = 19683$, que incluye tableros imposibles.



Con $n = 20$: $n^3 = 8000$, $2^n = 1048576$ y $n! \approx 2.4 \times 10^{18}$, donde $n! = n \cdot (n-1) \cdots 1$. Revisar todos los casos deja de ser viable.

Búsqueda completa e incompleta

Completa

- Revisa todo el espacio de forma sistemática.
- Encuentra la mejor solución o confirma que no existe.
- Suele ser muy costosa.

Incompleta

- Revisa solo una parte del espacio.
- No garantiza la mejor solución ni puede confirmar que no existe.
- Es rápida y se puede controlar con un presupuesto.

> Una búsqueda **constructiva** arma la solución desde cero, paso a paso. Una búsqueda **perturbativa** modifica una solución existente para mejorarla.

Qué significan P, NP y NP-hard

- P: problemas que se resuelven en tiempo polinomial. El tiempo crece como una potencia fija de n, como n^2 o n^3.
- NP (no determinista polinomial): problemas en que una solución propuesta se puede verificar en tiempo polinomial.
- NP-hard: problemas al menos tan difíciles como los más difíciles de NP. No es obligatorio que estén en NP.
- NP-completo: problemas que están en NP y además son NP-hard.

> Se asume $\mathrm{P} \neq \mathrm{NP}$. Por eso no se conoce un método rápido y exacto para los problemas NP-completos.

El problema del viajante (TSP)

El **TSP** (*Traveling Salesman Problem*, problema del viajante) pide visitar n ciudades una vez y volver a la ciudad de partida con el menor costo. Es un **CSOP** (problema de satisfacción y optimización de restricciones).

Decisión

¿Existe una ruta con costo menor o igual que K ? La respuesta es sí o no. Si se entrega una ruta, su costo se verifica rápido. Es NP-completo.

Optimización

¿Cuál es la ruta de menor costo? La respuesta es una ruta, no un sí o no. Es NP-hard.

Como recorrer una ruta al revés cuesta lo mismo, cada ruta se cuenta dos veces. El número de rutas distintas es $(n-1)!/2$.

- Con $n = 10$: $9!/2 = 181440$ rutas.
- Con $n = 20$: hay cerca de 6.1×10^{16} rutas.

Justificar una solución aproximada

> Una técnica aproximada no se justifica por ser factible. Se justifica por la calidad que alcanza frente a otras opciones.

- **Línea base:** una solución simple, como elegir al azar, para comparar.
- **Presupuesto:** el mismo número de evaluaciones o de tiempo para todos los métodos.
- **Repeticiones:** varias ejecuciones con distintas semillas. Una semilla es el número que fija el azar para repetir una ejecución. Se reportan el promedio y la dispersión.
- **Interpretación:** la diferencia frente a la línea base, en las unidades del costo.

Herramientas de Python

Herramienta	Qué aporta
scikit-fuzzy	Funciones de pertenencia, reglas y defusificación para sistemas difusos
DEAP (biblioteca de algoritmos evolutivos en Python)	Operadores y ciclo de algoritmos evolutivos
TensorFlow con Keras, o PyTorch	Construcción y entrenamiento de redes neuronales

Un mismo trabajo de redes neuronales usa una sola de estas bibliotecas.

Sistemas difusos

Gradualidad, reglas y el método Mamdani

Conjuntos difusos y operaciones

Conjunto clásico

Un elemento pertenece (valor 1) o no pertenece (valor 0).

Conjunto difuso

Cada elemento tiene un grado de pertenencia entre 0 y 1, útil para conceptos graduales como "tibio": $\mu_A(x) \in [0, 1]$.

- **Intersección ("y"):** mínimo, $\mu_{A \cap B} = \min(\mu_A, \mu_B)$.
- **Unión ("o"):** máximo, $\mu_{A \cup B} = \max(\mu_A, \mu_B)$.
- **Complemento ("no"):** $\mu_{\bar{A}} = 1 - \mu_A$.

> Con $28\,^\circ\mathrm{C}$, $\mu_{\text{conf}} = 0.7$ y $\mu_{\text{alta}} = 0.3$. Entonces "confortable y alta" vale $\min(0.7, 0.3) = 0.3$.

Formas de las funciones de pertenencia

- **Triangular:** sube en línea recta desde a hasta b, llega a 1 en b y baja hasta c.
- **Trapezoidal:** igual que la triangular, pero con una meseta de grado 1 entre b y c.
- **Gaussiana:** tiene forma de campana, centrada en m, con un ancho σ.

Con una función triangular de $a = 15$, $b = 25$ y $c = 35$, el grado en $x = 28$ está en la parte descendente:

$$\mu(28) = \frac{c - x}{c - b} = \frac{35 - 28}{35 - 25} = 0.7$$

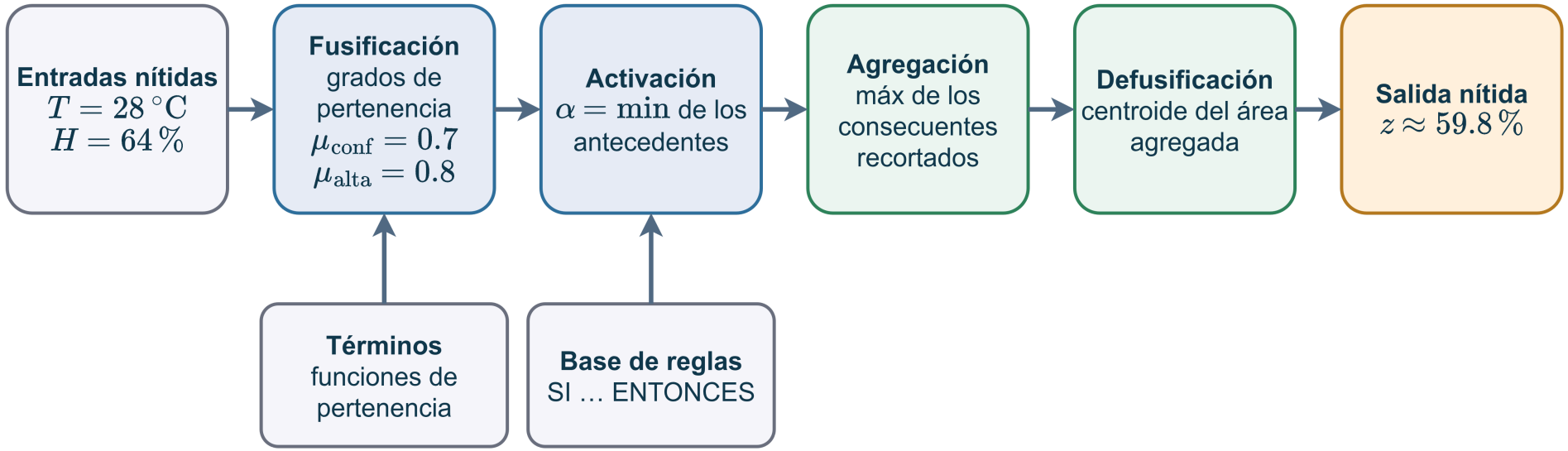
Variables lingüísticas y reglas

- Una **variable lingüística** es una magnitud, como la temperatura, con términos como baja, confortable o alta. Cada término tiene su propia función de pertenencia.
- Una **regla** tiene la forma SI … ENTONCES. Por ejemplo: SI la temperatura es alta Y la humedad es alta, ENTONCES la ventilación es alta.
- La "Y" se evalúa con el mínimo, y la "O" con el máximo.

> Con 3 términos de temperatura y 2 de humedad, hay $3 \times 2 = 6$ combinaciones posibles de condiciones.

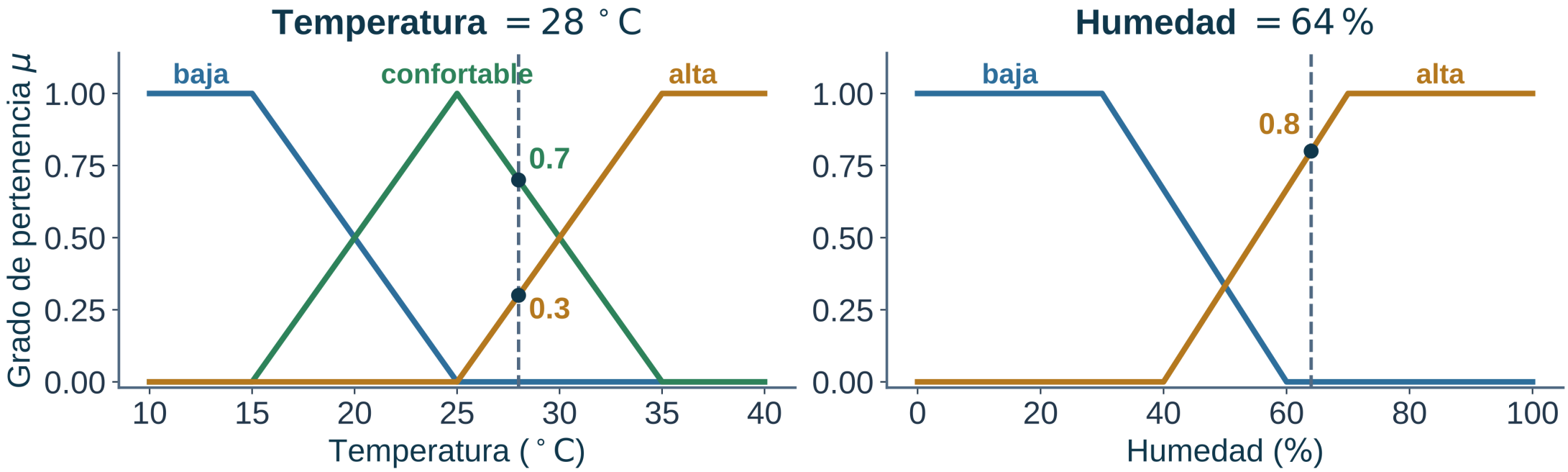
Etapas de un sistema Mamdani

El método Mamdani es un sistema difuso con reglas SI … ENTONCES y salida numérica.



Las entradas exactas se convierten en grados de pertenencia. Las reglas se activan, sus resultados se combinan y el centroide entrega un número.

Fusificación: de número a grados



Fusificar es calcular el grado de pertenencia de cada entrada a cada término. Con $T = 28\,^{\circ}\mathrm{C}$ (temperatura): confortable 0.7, alta 0.3 y baja 0. Con $H = 64\,\%$ (humedad): alta 0.8 y baja 0.

Activación de reglas

Cada regla combina sus condiciones con el mínimo. El resultado, α, indica qué tanto se cumple la regla. Ese valor recorta el conjunto de salida a esa altura.

Solo dos reglas tienen $\alpha > 0$:

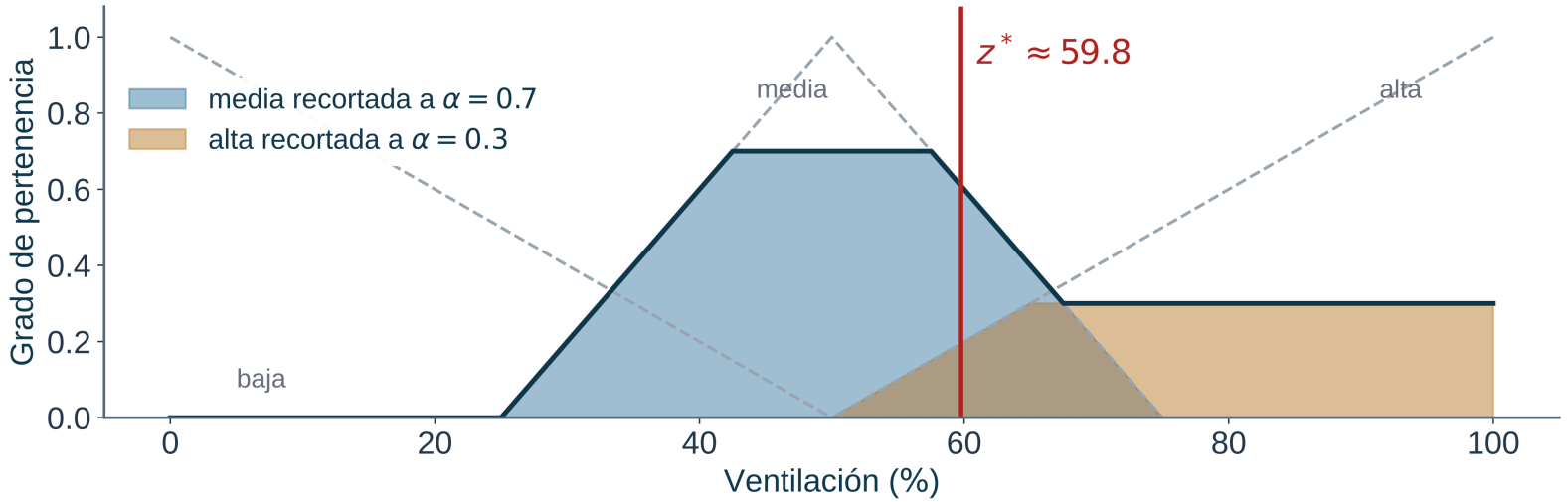
- SI T es confortable Y H es alta, ENTONCES la ventilación es media: $\alpha = \min(0.7, 0.8) = 0.7$.
- SI T es alta Y H es alta, ENTONCES la ventilación es alta: $\alpha = \min(0.3, 0.8) = 0.3$.

Las demás reglas tienen una condición con grado 0, así que su α es 0.

Agregación y defusificación

La **agregación** une los conjuntos recortados con el máximo. La **defusificación** convierte el resultado en un número: el **centroide**, el punto que equilibra el área:

$$z^* = \frac{\int z\,\mu(z)\,dz}{\int \mu(z)\,dz}$$



El centroide da una ventilación de $z^* \approx 59.8\,\%$.

Validación del sistema difuso

- Revisar el rango y las unidades de cada variable.
- Revisar que cada valor posible de entrada tenga al menos un término con grado mayor que 0.
- Revisar que las reglas no se contradigan.
- Probar los valores extremos y los bordes del rango.
- Ver si el resultado cambia mucho con otro método de defusificación.

> Un sistema difuso se entiende y se defiende solo si sus términos y reglas tienen un significado claro.

Computación evolutiva

Población, operadores y dos objetivos

El TSTP: TSP con dos objetivos

El **TSP** (problema del viajante) pide visitar n ciudades una vez y volver a la de partida con el menor costo. El **TSTP** (*Traveling Salesman Trolley Problem*) es su variante de dos objetivos. Cada arco, es decir, el tramo entre dos ciudades, tiene además un número entero de personas afectadas, entre 1 y 5. Su nombre viene del dilema del tranvía, un problema ético sobre decisiones que dañan a personas.

Se minimizan dos objetivos a la vez:

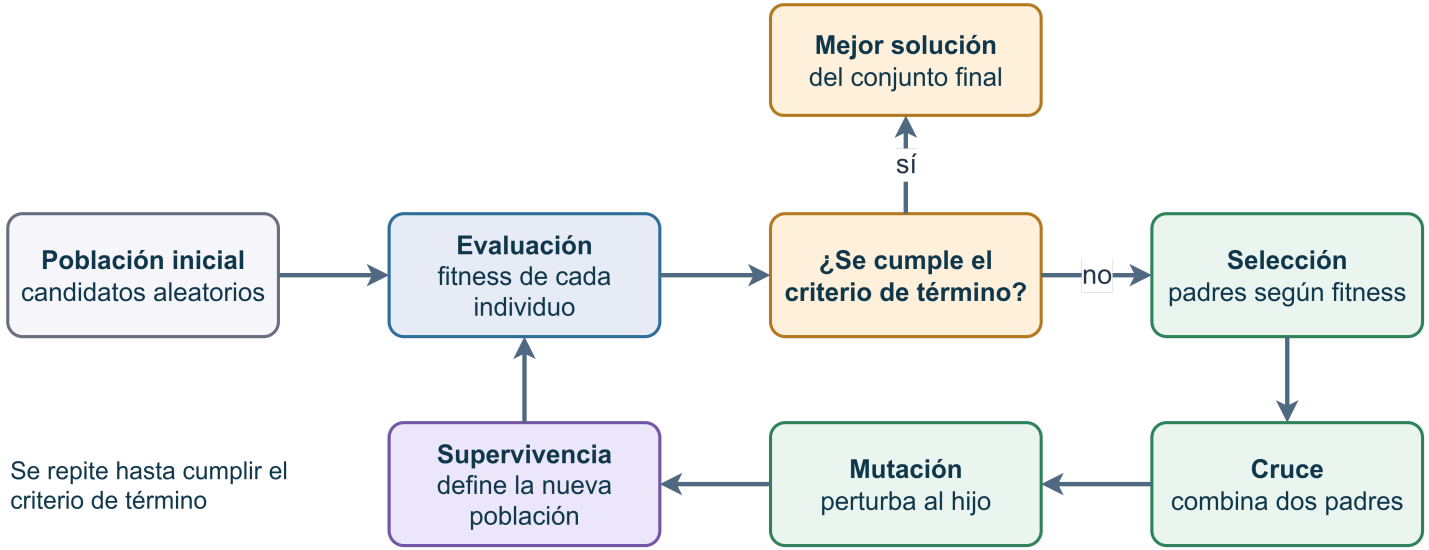
- La distancia total L de la ruta.
- El total de personas afectadas M en la ruta.

> Enumerar todas las rutas no es viable. Por eso se usa una **población** de rutas que se mejora generación tras generación. Los dos objetivos pueden entrar en conflicto.

Conceptos básicos de algoritmos evolutivos

- **Individuo:** un candidato a solución. En el TSTP, una ruta, que se representa como una **permutación**: un orden de todas las ciudades.
- **Población:** conjunto de individuos que se evalúan en una misma generación.
- **Generación:** un ciclo completo de evaluación, selección, cruce, mutación y supervivencia.
- **Fitness:** puntaje que mide la calidad de un individuo. Aquí se minimiza.
- **Padres e hijos:** los padres son los individuos elegidos para reproducirse, y los hijos son sus descendientes.
- **Supervivencia:** decide qué individuos pasan a la siguiente generación. El **elitismo** conserva siempre al mejor.
- **Explotación y exploración:** buscar cerca de las buenas soluciones o en zonas nuevas. Si la explotación es excesiva, se cae en un **óptimo local**, es decir, una solución mejor que sus vecinas, pero no la mejor posible.
- **Criterio de término:** condición que detiene el algoritmo.

Ciclo de un algoritmo evolutivo



La población se evalúa. Si no se cumple el criterio de término, se generan hijos con cruce y mutación, y la supervivencia forma la nueva población.

> Sobre permutaciones, el cruce y la mutación deben conservar un orden válido de ciudades.

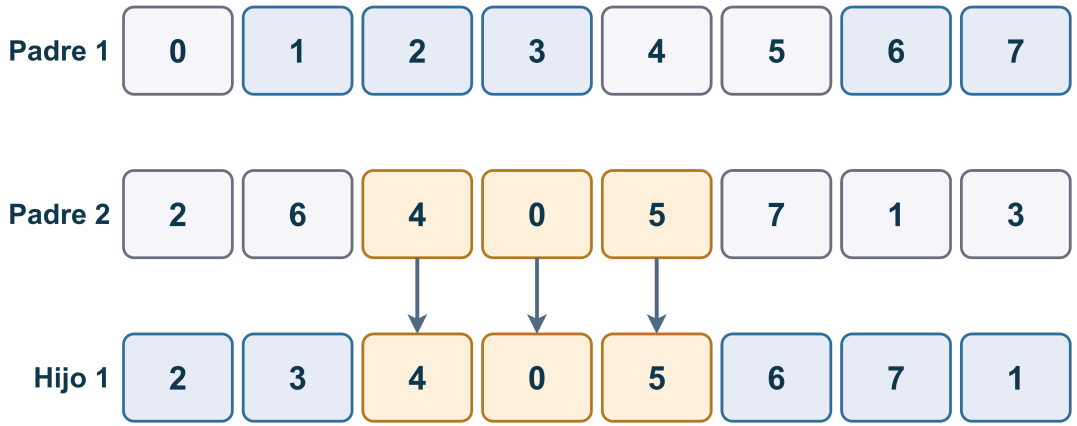
Selección y mutación

- **Selección:** elige individuos según su fitness. Por torneo, se enfrentan grupos pequeños y gana el de mejor fitness. Un torneo grande favorece a los mejores, pero reduce la diversidad.
- **Mutación:** cambia un poco un individuo. En una permutación, puede intercambiar dos ciudades o invertir un tramo.
- **Diversidad:** variedad de individuos en la población. La mutación ayuda a mantenerla y evita que todos se parezcan.

> Un cruce cualquiera podría repetir ciudades, y eso invalidaría la ruta. Para permutaciones se usa el cruce de orden (OX).

Cruce de orden (OX)

El **cruce de orden** (OX) combina dos padres sin repetir ciudades. Copia un tramo de un padre en las mismas posiciones y completa el resto con el orden del otro padre. Así el hijo sigue siendo una ruta válida.



Las ciudades se numeran desde 0. El tramo de las posiciones 2 a 4 del Padre 2 es $4, 0, 5$. Las demás posiciones se llenan con $6, 7, 1, 2, 3$, en el orden del Padre 1 y desde la posición 5.

Familias de algoritmos evolutivos

Familia	Representación	Operadores	Aplicación
EP (programación evolutiva)	Vectores	Mutación	Siempre
ES (estrategias evolutivas)	Vectores reales	Mutación y cruce	Siempre
GA (algoritmo genético)	Cadenas o permutaciones	Cruce y mutación	Con probabilidad
GP (programación genética)	Árboles	Subárboles y mutación	Con probabilidad

En EP y ES, la mutación y el cruce siempre se aplican. En GA y GP se aplican con probabilidad: p_c para el cruce y p_m para la mutación.

Las rutas del TSTP son permutaciones, así que un GA con cruce OX encaja directamente.

Fitness con dos objetivos

El **fitness** resume L y M en un solo puntaje que se minimiza. Cada valor se divide por una referencia para que ambos queden en escalas comparables. El peso α, entre 0 y 1, decide cuánto pesa cada objetivo:

$$f_\alpha = \alpha \, \frac{L}{L_{ref}} + (1 - \alpha) \, \frac{M}{M_{ref}}$$

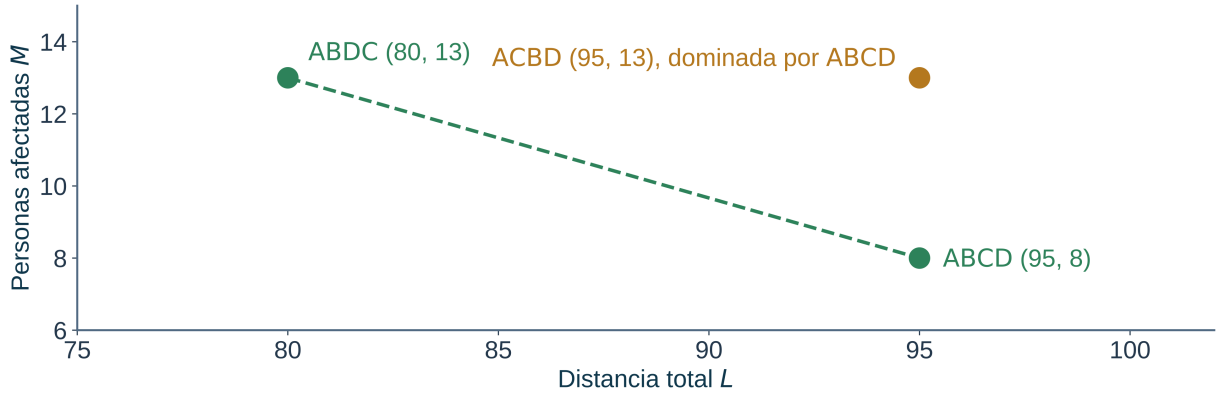
La elección de α debe justificarse. Por ejemplo, ABCD tiene $L = 10 + 35 + 30 + 20 = 95$ y $M = 2 + 3 + 2 + 1 = 8$. Con las tres rutas de la tabla, $L_{ref} = 90$ y $M_{ref} = 34/3 \approx 11.33$.

Ruta	L	M	$f_{0.5}$	$f_{0.9}$
ABCD	95	8	0.8807	1.0206
ABDC	80	13	1.0180	0.9147
ACBD	95	13	1.1013	1.0647

Con $\alpha = 0.5$ gana ABCD. Con $\alpha = 0.9$, que pesa más la distancia, gana ABDC.

Frente de Pareto

Una solución **domina** a otra si es igual o mejor en todos los objetivos y estrictamente mejor en uno. Las soluciones que nadie domina forman el **frente de Pareto**: cada una es un compromiso distinto entre L y M.



ACBD es dominada por ABCD, que tiene la misma distancia y menos personas afectadas. ABCD y ABDC no se dominan entre sí.

> Con un α fijo, el fitness elige un solo punto del frente.

Término y comparación de algoritmos

El criterio de término puede ser un número de generaciones, un número de evaluaciones de fitness, un tiempo máximo o un número de generaciones sin mejora.

Para comparar dos algoritmos se usan al menos tres criterios: el mejor fitness, el número de evaluaciones, el tiempo, la memoria o la estabilidad entre semillas.

Con $\mu = 100$ padres y $\lambda = 100$ hijos por generación, la población inicial se evalúa una vez y cada una de las 500 generaciones evalúa λ hijos:

$$N_{eval} = \mu + 500\,\lambda = 100 + 500 \cdot 100 = 50100$$

Al igual que al justificar una aproximación, ambos algoritmos deben usar el mismo presupuesto de evaluaciones para que la comparación sea justa.

Redes neuronales

Entrenamiento, particiones y redes convolucionales

Aprendizaje supervisado

Un modelo aprende a partir de ejemplos con respuesta conocida. Recibe una entrada x, produce una predicción $\hat{y}$ y ajusta sus parámetros θ para que $\hat{y}$ se acerque a la respuesta real y. La **pérdida** mide qué tan mal predice el modelo, y entrenar significa reducirla.

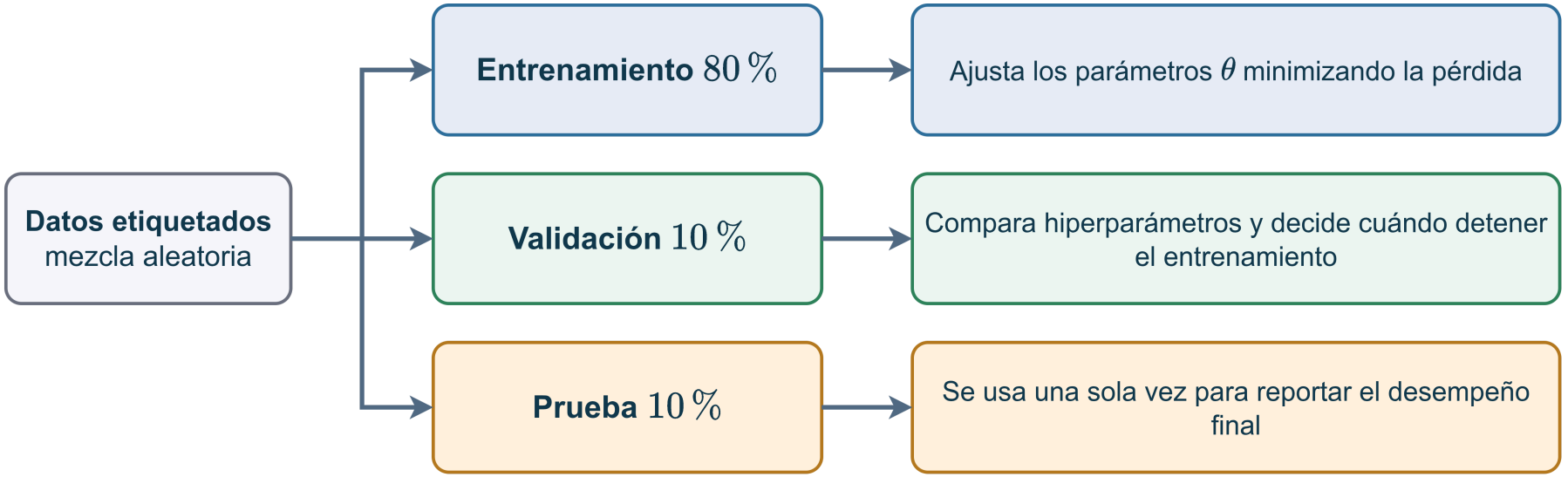
Clasificación

- Salida: una categoría, por ejemplo abeja, mosca o escarabajo.
- Pérdida: entropía cruzada.
- Métricas: exactitud, precisión, recall y F_1.

Regresión

- Salida: una cantidad numérica, por ejemplo un precio.
- Pérdida: error cuadrático medio (MSE).

Particiones de los datos



- **Entrenamiento ($80\,\%$):** ajusta los parámetros θ.
- **Validación ($10\,\%$):** compara configuraciones y decide cuándo detener el entrenamiento.
- **Prueba ($10\,\%$):** se usa una sola vez para reportar el desempeño final.

Hay fuga de información si alguna decisión usa datos de prueba. Por ejemplo, el preprocesamiento se ajusta solo con los datos de entrenamiento.

Matriz de confusión

	Clase predicha A	Clase predicha B	Clase predicha C	Recall
Real A	18	1	1	0.90
Real B	2	15	3	0.75
Real C	0	2	18	0.90
Precisión	0.90	0.83	0.82	Exactitud 0.85

Las filas son la clase real y las columnas la clase predicha. La diagonal reúne los aciertos. Los errores quedan fuera de la diagonal y muestran qué clase se confunde con cuál.

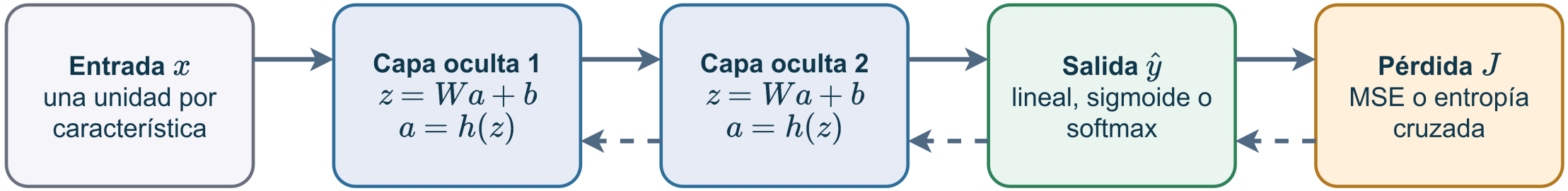
Métricas de clasificación

- **Exactitud:** proporción de aciertos sobre el total.
- **Precisión:** de los casos predichos como una clase, qué proporción era correcta.
- **Recall** (sensibilidad): de los casos reales de una clase, qué proporción se detectó.
- F_1: promedio armónico de P y R, con $F_1 = \dfrac{2PR}{P+R}$. Es alto solo si ambos son altos.

- Exactitud, con $N = 60$ casos: $(18 + 15 + 18)/60 = 0.85$.
- Clase B: $P = 15/18 = 0.8333$ y $R = 15/20 = 0.75$.
- $F_1 = \dfrac{2 \cdot 0.8333 \cdot 0.75}{0.8333 + 0.75} = 0.7895.$

Con clases desbalanceadas, la exactitud puede parecer alta aunque una clase casi no se detecte.

Neurona y red multicapa



Cada capa calcula $z = Wa + b$, con pesos W, sesgos b y entradas a de la capa anterior. Luego aplica una activación $a = h(z)$. Una red multicapa (MLP, *multilayer perceptron*) apila varias de estas capas.

La línea continua muestra la predicción. La línea punteada muestra cómo el error se devuelve para ajustar los pesos. Sin una activación no lineal, varias capas equivalen a una sola.

Funciones de activación

- **ReLU** (*rectified linear unit*): $h(z) = \max(0, z)$. Se usa en capas ocultas.
- **Sigmoide**: $h(z) = \dfrac{1}{1 + e^{-z}}$, con valores entre 0 y 1. Se usa para clasificación binaria.
- **Tangente hiperbólica**: valores entre -1 y 1.
- **Softmax**: convierte los puntajes de K clases en probabilidades que suman 1. Se usa para clasificación con varias clases.
- **Lineal**: $h(z) = z$. Se usa en la salida de regresión.

> La activación de salida y la pérdida se eligen juntas. Softmax se combina con entropía cruzada, y la salida lineal con MSE.

Softmax y entropía cruzada

Softmax transforma los puntajes z_k en probabilidades:

$$p_k = \frac{e^{z_k}}{\sum_j e^{z_j}}$$

La entropía cruzada mide la pérdida como $L = -\ln p_y$, donde p_y es la probabilidad que el modelo asigna a la clase correcta. Cuanto mayor sea esa probabilidad, menor es la pérdida.

Con puntajes $z = (2.0, 1.0, 0.1)$:

- Exponenciales: $e^z = (7.3891, 2.7183, 1.1052)$, con suma 11.2125.
- Probabilidades: $p = (0.6590, 0.2424, 0.0986)$.
- Si la clase correcta es la 1: $L = -\ln 0.6590 = 0.417$.
- Si la clase correcta es la 2: $L = -\ln 0.2424 = 1.417$.

Entrenamiento por gradiente

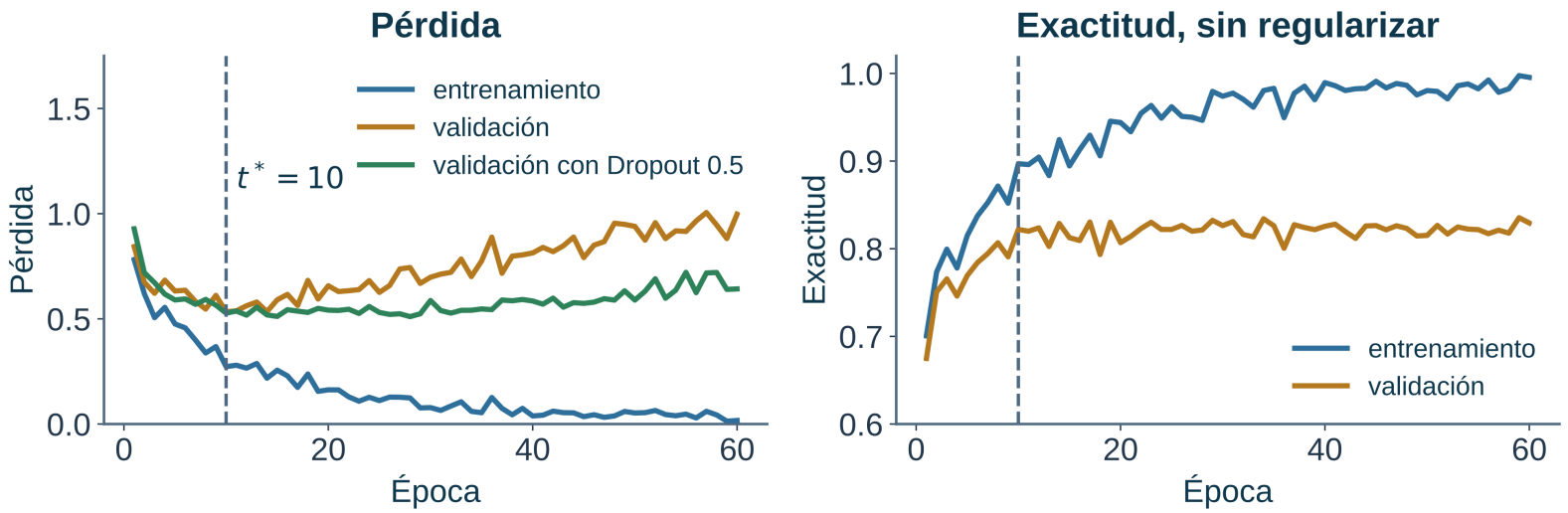
Los parámetros se actualizan con:

$$\theta \leftarrow \theta - \eta \nabla_\theta J$$

- J: pérdida. $\nabla_\theta J$ indica hacia dónde sube la pérdida, así que el paso va en sentido contrario.
- η: tasa de aprendizaje, es decir, el tamaño del paso.
- **Retropropagación:** calcula ese gradiente capa por capa, desde la salida hacia atrás.
- **Minilote:** grupo pequeño de ejemplos para cada paso. **Época:** una pasada completa por todos los datos.
- **SGD** (descenso por gradiente estocástico) usa el gradiente directo. **Adam** ajusta el tamaño del paso para cada parámetro.

> Con $n = 4000$ ejemplos y minilotes de 32, hay $4000/32 = 125$ pasos por época. En 10 épocas son $10 \cdot 125 = 1250$ pasos.

Subajuste y sobreajuste



Si el modelo memoriza el entrenamiento, su pérdida de validación sube. Eso es **sobreajuste**. Si ni siquiera aprende bien el entrenamiento, es **subajuste**.

Ejemplo: red con dos capas ocultas de 512 neuronas, 2000 ejemplos de entrenamiento y Fashion-MNIST (imágenes de prendas en 10 clases). La pérdida de validación es mínima en la época $t^* = 10$, con 0.534, y termina en 0.998.

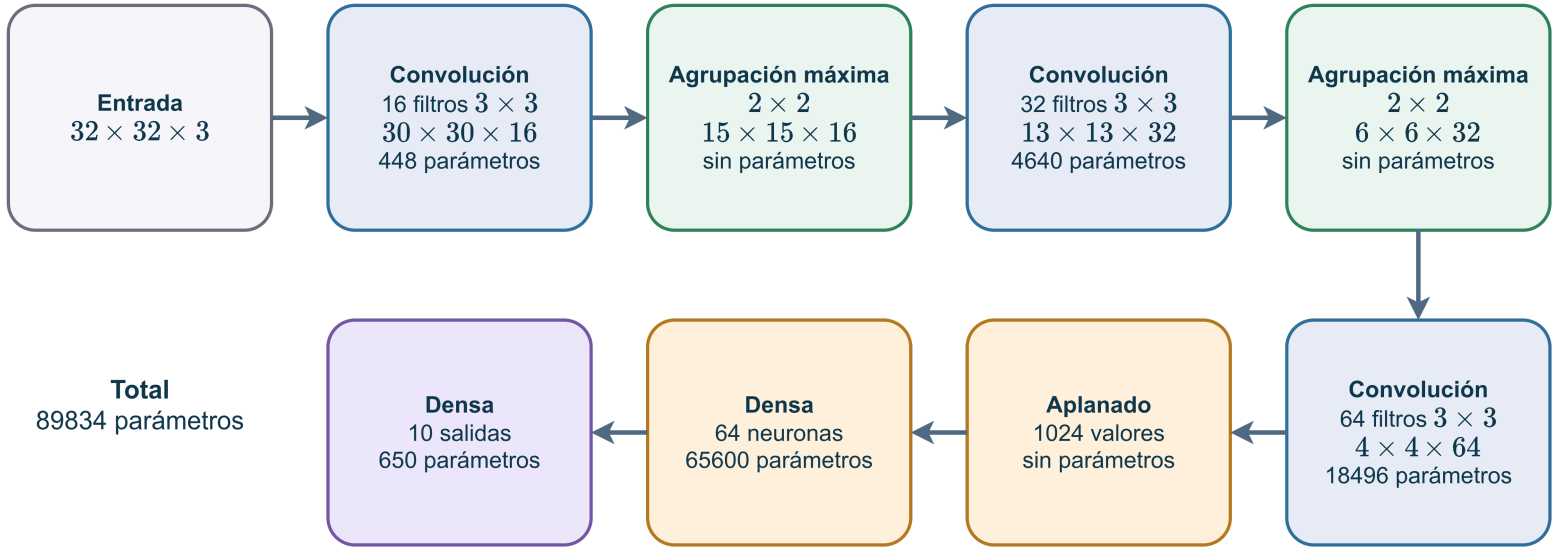
Regularización

- **Penalización L_2:** suma al costo $\lambda \sum_i w_i^2$, lo que mantiene los pesos pequeños.
- **Dropout:** apaga neuronas al azar durante el entrenamiento, con probabilidad p. Así la red no depende de pocas neuronas.
- **Parada temprana:** detiene el entrenamiento en la época t^*, donde la pérdida de validación es mínima.

> Con Dropout de 0.5 en la misma red, la pérdida de validación termina en 0.642, frente a 0.998 sin regularización.

Red convolucional (CNN)

Una red convolucional (CNN, *convolutional neural network*) recorre la imagen con filtros pequeños para detectar patrones locales, como bordes. La **agrupación máxima** reduce el tamaño de la imagen y deja el valor más alto de cada bloque. Al final, las capas **densas** clasifican.



Cada filtro se reutiliza en toda la imagen, por eso la CNN tiene menos parámetros que una red totalmente conectada.

Diseño experimental y comparación

- Se usan las mismas particiones y una sola biblioteca para todos los modelos.
- La partición de prueba se usa una vez, al final.
- Los modelos se comparan con varias semillas, no con una sola ejecución.

> Una diferencia pequeña entre dos modelos puede deberse al azar. Por eso se reportan varias ejecuciones con su dispersión.

Referencias

1. L. A. Zadeh, "Fuzzy Sets", *Information and Control*, 1965.
2. L. A. Zadeh, "Fuzzy Logic, Neural Networks, and Soft Computing", *Communications of the ACM*, 1994.
3. A. M. Turing, "On Computable Numbers, with an Application to the Entscheidungsproblem", 1936.
4. S. A. Cook, "The Complexity of Theorem-Proving Procedures", 1971.
5. R. M. Karp, "Reducibility Among Combinatorial Problems", 1972.
6. T. J. Ross, *Fuzzy Logic with Engineering Applications*, 3.ª ed., Wiley, 2010.
7. D. E. Goldberg, *Genetic Algorithms in Search, Optimization, and Machine Learning*, Addison-Wesley, 1989.
8. I. Goodfellow, Y. Bengio y A. Courville, *Deep Learning*, MIT Press, 2016.